

2

CONTROL FLOW STATEMENTS

2.1 Switch Statement

- “Case Value” can be of character type and int type.
- There can be one or many number of cases.
- Statements associated with a matching case executes until a break statement is reached.
- The position of default case does not matter. It can be placed anywhere.
- Default case is optional.
- Duplicate case labels are not allowed.
- Statements written before all the cases are ignored by the compiler.
- The break statement is optional.
- Case label cannot be a variable.

2.2 Iterative statements (Loop)

An iterative statement, or loop, repeatedly executes a group of statements, known as body of loop, until the controlling expression is false.

2.2.1 For Loop

- The for statement evaluates 3 expressions and executes the loop body until second controlling expression executes to false.
- It is recommended to use for loop when the number of iterations is known in advance.
- Syntax of for loop is:
for (expression – 1(optional); expression – 2(optional); expression – 3(optional))
{
 statement – 1
 statement – 2
 :
 statement – n
}
 - for loop executes the loop body 0 or more times.
 - for statements works as follows:
 - (a) expression – 1 is evaluated once before the first iteration of the loop.
 - (b) expression – 2 is a expression that determines whether to terminate the loop. expression – 2 is evaluated before every iteration.
If the expression is true (non - zero), the loop body executed.
If the expression is false (zero), execution of the for statement is terminated.
 - (c) expression – 3 is evaluated after each iteration.

(d) The for statement executes until expression-2 is false (0), or until a jump statement terminates execution of the loop.

- All 3 expression can be omitted

(a) If we omit expression-2, the condition is considered as true for an example:

```
for (i = 0; ; i++) } represent an infinite loop
statement.
```

2.3 While Statement

- The while statement evaluates a controlling expression before every execution of the loop body.
- If the controlling expression is true (non-zero), the loop body is executed. If the controlling expression is false (zero), then the while statement terminates.
- Syntax:

```
while (expression)
statement
OR
while (expression)
{
statement-1
statement-2
:
statement-n
}
```

2.4 Do-while Loops

- Both for and while loop checks the loop termination condition before every iteration but do while loop check the condition after executing the loop body.
- do while loop body executes at least 1 time, no matter whether the loop termination condition is false or true.
- Syntax is:

```
do
statement
while (expression);
OR
do {
statement-1
statement-2
:
statement-n
} while (expression);
```

2.5 Break Statement

- The break statement provides an early exit from for, while and do while.
- A break statement causes the innermost loop or switch to be exited immediately.
- In implementation, when we know the maximum number of repetition but some condition is there, where we require to terminate the repetition process, then use break statement.

Example:

```
void main ()
{
int i = 1;
while (i <= 15)
{
```

```
printf("%d\t",i);
if (i > 4)
    break;
i = i + 1;
}
printf("End");
}
```

Output: 1 2 3 4 End

In above code, when i becomes 5, the condition $i > 4$ becomes true, so break statement executes & control passed outside of the loop body i.e., `printf("End")` executed.

2.6 Continue Statement

- Continue statement is related to break but it is not used that much frequently.
- Whenever continue is encountered in a loop, it skips the remaining statement of the current iteration and continues with the next iteration of the loop.
- Only applicable with loops, not with switch statement.

Example:

```
void main () {
    int i = 1;
    for (i = 1; i <= 10; i++)
    {
        if(i%2 == 0)
            continue;
        printf("%d",i);
    }
}
```

Output: 13579 (All odd numbers between 1 to 10)

whenever i becomes even, condition $i\%2 == 0$ becomes true & continue causes the control to go to $i++$ & `printf` is skipped.

